



PROJET RCP211
RAPPORT DE PROJET
IA POUR LES DONNÉES MULTIMÉDIA

Inverse Reinforcement Learning

Étudiant :
Fayz EL RAZAZ

6 juin 2026

Table des matières

I	Introduction	2
II	Démarche proposée et environnement d'étude	3
II.1	Choix méthodologique	3
II.2	Principe général de la démarche	3
II.3	Présentation de l'environnement CartPole	4
II.4	Nature des démonstrations expertes	5
III	Construction des démonstrations expertes	5
III.1	Objectif	5
III.2	Entraînement d'un agent expert	5
III.3	Génération des démonstrations	6
III.4	Filtrage des trajectoires expertes	6
III.5	Construction du dataset final	7
IV	Modélisation de la fonction de récompense	7
IV.1	Difficulté de la modélisation	8
IV.2	Première approche : apprentissage discriminatif état par état	8
IV.3	Limite de l'approche discriminative	9
V	Méthode retenue : apprentissage par préférence de trajectoires	9
V.1	Principe	9
V.2	Boucle d'apprentissage	10
V.3	Architecture de la récompense	10
VI	Résultats expérimentaux	11
VI.1	Résultats de la boucle par préférence de trajectoires	11
VI.2	Réutilisation de la récompense finale	11
VI.3	Analyse	12
VII	Conclusion	12

I Introduction

L'apprentissage par renforcement (*Reinforcement Learning*, RL) est un domaine de l'intelligence artificielle dans lequel un agent apprend à interagir avec un environnement afin de maximiser une récompense cumulative [7]. À chaque instant, l'agent observe un état de l'environnement, choisit une action, puis reçoit une récompense indiquant la qualité de cette action relativement à l'objectif recherché. Progressivement, l'agent apprend une politique de décision lui permettant d'adopter un comportement performant.

Cette approche a permis d'obtenir des résultats remarquables dans de nombreux domaines tels que les jeux, la robotique ou encore l'optimisation de processus complexes. Les avancées récentes du *Deep Reinforcement Learning* ont notamment permis d'atteindre des performances proches, voire supérieures, à celles de l'humain dans certains environnements complexes [4, 6]. Toutefois, une difficulté majeure du RL réside dans la définition de la fonction de récompense. Dans de nombreux problèmes réels, il est difficile de formaliser explicitement ce qu'est un « bon » comportement. Une récompense mal définie peut conduire l'agent à adopter des stratégies inattendues ou inefficaces.

L'*Inverse Reinforcement Learning* (IRL), également appelé apprentissage inverse par renforcement, propose une approche différente de ce problème [5, 1]. Plutôt que de définir manuellement une fonction de récompense, l'idée est d'observer un comportement expert et d'en déduire la récompense implicite qui semble motiver ce comportement. L'objectif n'est donc plus directement d'apprendre une politique optimale, mais d'apprendre une fonction de récompense cohérente avec les démonstrations observées.

Dans un cadre réel, ces démonstrations peuvent être fournies par un humain réalisant une tâche considérée comme optimale ou experte. Par exemple, dans un contexte robotique, un humain peut guider un bras robotisé afin de montrer le comportement attendu. L'algorithme d'IRL cherche alors à extraire les caractéristiques implicites de ce comportement afin de reconstruire une fonction de récompense permettant ensuite à un agent d'apprendre de manière autonome.

Parmi les approches majeures de l'IRL, les méthodes basées sur le principe de maximum d'entropie occupent une place importante [9, 8]. Ces méthodes considèrent que les démonstrations expertes correspondent à des trajectoires plus susceptibles d'apparaître car elles minimisent implicitement un coût ou maximisent une récompense inconnue. Plus récemment, Finn, Levine et Abbeel ont proposé une extension profonde de cette approche à travers le papier *Guided Cost Learning : Deep Inverse Optimal Control via Policy Optimization* [3], permettant d'apprendre des fonctions de coût complexes à l'aide de réseaux de neurones.

Dans le cadre de ce projet, nous nous intéressons à une version simplifiée de ce problème en utilisant l'environnement *CartPole* de la bibliothèque *Gymnasium* [2]. Afin de nous concentrer sur les mécanismes fondamentaux de l'IRL sans introduire la complexité d'un système physique réel, les démonstrations expertes ne seront pas produites par un humain mais par un agent préalablement entraîné à résoudre la tâche de manière performante. Cet agent jouera ainsi le rôle d'un expert dont nous chercherons à reproduire implicitement les objectifs à travers l'apprentissage d'une fonction de récompense.

Bien que notre implémentation soit volontairement simplifiée, les principes fondamentaux étudiés dans le travail de Finn et al. [3] demeurent les mêmes :

- observer un comportement expert ;
- apprendre une récompense cohérente avec ce comportement ;
- puis réentraîner un agent à partir de cette récompense apprise.

L'objectif du projet est donc double :

- comprendre les mécanismes théoriques de l'Inverse Reinforcement Learning ;
- mettre en œuvre une chaîne complète d'apprentissage de récompense sur un environnement simple et contrôlé.

II Démarche proposée et environnement d'étude

II.1 Choix méthodologique

L'objectif principal de ce projet est de comprendre les mécanismes fondamentaux de l'Inverse Reinforcement Learning dans un cadre suffisamment simple pour permettre une analyse détaillée des différentes étapes de l'apprentissage. Le papier de Finn, Levine et Abbeel propose une approche très avancée appliquée à des systèmes robotiques continus de grande dimension, utilisant des réseaux de neurones profonds et des techniques complexes d'optimisation de politiques. Reprendre l'intégralité de cette architecture dépasserait largement le cadre du projet et introduirait une complexité importante rendant l'analyse pédagogique plus difficile.

Nous avons donc choisi d'adopter une approche simplifiée reposant sur l'environnement `Cart Pole` de la bibliothèque `Gymnasium`. Ce choix permet de conserver les concepts centraux de l'IRL tout en travaillant dans un environnement :

- simple à visualiser ;
- rapide à entraîner ;
- suffisamment riche pour illustrer les mécanismes de l'apprentissage ;
- largement utilisé comme environnement de référence en apprentissage par renforcement.

L'objectif n'est donc pas ici de reproduire toute la complexité du papier étudié, mais plutôt d'en comprendre les principes fondamentaux à travers une implémentation simplifiée et contrôlée.

II.2 Principe général de la démarche

Dans un problème classique d'apprentissage par renforcement, la fonction de récompense est connue. L'agent interagit avec l'environnement et apprend progressivement une politique lui permettant de maximiser cette récompense.

Dans le cadre de l'Inverse Reinforcement Learning, la situation est inversée :

- nous observons un comportement expert ;
- mais la fonction de récompense qui motive ce comportement est inconnue.

L'objectif devient alors d'apprendre une fonction de récompense cohérente avec les démonstrations observées.

Dans un contexte réel, ces démonstrations seraient généralement fournies par un humain expert réalisant la tâche considérée comme optimale. Par exemple, dans un contexte robotique, un opérateur humain pourrait guider un robot afin de lui montrer le comportement attendu. L'algorithme chercherait ensuite à extraire les caractéristiques implicites de ce comportement afin de reconstruire une fonction de récompense adaptée.

Dans le cadre de ce projet, nous remplaçons cet expert humain par un agent déjà entraîné à résoudre efficacement la tâche. Cette simplification permet de disposer de démonstrations expertes cohérentes et reproductibles tout en conservant exactement la logique théorique de l'IRL.

La démarche générale du projet peut ainsi être résumée en plusieurs étapes :

1. entraîner un agent expert sur l'environnement `CartPole` à l'aide d'un algorithme classique de Reinforcement Learning ;
2. générer plusieurs démonstrations expertes à partir de cet agent ;
3. utiliser ces démonstrations pour apprendre une fonction de récompense ;
4. réentraîner un nouvel agent en utilisant uniquement la récompense apprise ;
5. comparer le comportement obtenu avec celui de l'expert initial.

Cette approche permet d'évaluer si la récompense apprise capture correctement les caractéristiques importantes du comportement expert.

II.3 Présentation de l'environnement `CartPole`

L'environnement `CartPole` est un problème classique d'apprentissage par renforcement dans lequel un agent doit maintenir un mât vertical en contrôlant un chariot mobile.

Le système est constitué :

- d'un chariot pouvant se déplacer horizontalement ;
- d'un pendule fixé sur ce chariot ;
- d'un agent pouvant appliquer une force vers la gauche ou vers la droite.

L'objectif est de maintenir le pendule en équilibre le plus longtemps possible sans que :

- l'angle du mât ne dépasse une certaine limite ;
- ou que le chariot ne sorte de la zone autorisée.

À chaque instant, l'environnement fournit un état composé de quatre variables :

$$s_t = (x, \dot{x}, \theta, \dot{\theta})$$

où :

- x représente la position horizontale du chariot ;
- $\dot{x}$ représente la vitesse du chariot ;
- θ représente l'angle du pendule ;
- $\dot{\theta}$ représente la vitesse angulaire du pendule.

L'espace d'action est discret :

$$a_t \in \{0, 1\}$$

avec :

- 0 : pousser le chariot vers la gauche ;
- 1 : pousser le chariot vers la droite.

Une trajectoire complète correspond alors à une succession d'états et d'actions :

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$$

Dans le cadre de l'IRL, ces trajectoires expertes constituent les données principales utilisées pour apprendre une fonction de récompense.

II.4 Nature des démonstrations expertes

Les démonstrations expertes correspondent aux trajectoires produites par l'agent préalablement entraîné. Chaque démonstration contient l'ensemble des transitions observées au cours d'un épisode :

$$(s_t, a_t, s_{t+1})$$

ou, de manière plus complète :

$$(s_t, a_t, r_t, s_{t+1}, done_t)$$

Même si la récompense réelle de l'environnement n'est pas utilisée directement pour l'apprentissage inverse, elle peut être conservée à des fins d'analyse ou de validation.

L'idée fondamentale est que l'algorithme d'IRL ne cherche pas à mémoriser directement les actions de l'expert, mais à comprendre quelles caractéristiques des états semblent implicitement valorisées par ce comportement.

Autrement dit, l'objectif n'est pas d'apprendre :

« dans cet état, il faut aller à gauche »

mais plutôt :

« certains états semblent meilleurs que d'autres selon les démonstrations observées »

Cette distinction constitue l'un des éléments centraux de l'Inverse Reinforcement Learning.

III Construction des démonstrations expertes

III.1 Objectif

Avant de pouvoir apprendre une fonction de récompense par Inverse Reinforcement Learning, il est nécessaire de disposer de démonstrations expertes. Ces démonstrations constituent les exemples de comportements considérés comme performants à partir desquels l'algorithme tentera d'inférer une récompense implicite.

Dans un contexte réel, ces démonstrations pourraient être produites par un humain expert réalisant correctement une tâche donnée. Cependant, afin de simplifier l'expérimentation et de garantir des comportements cohérents et reproductibles, nous utilisons ici un agent préalablement entraîné sur l'environnement `CartPole`. Cet agent jouera le rôle de l'expert dont nous chercherons à reproduire implicitement les objectifs.

III.2 Entraînement d'un agent expert

Pour produire ces démonstrations, nous avons utilisé l'algorithme *Proximal Policy Optimization* (PPO), implémenté dans la bibliothèque `Stable-Baselines3`. PPO est un algorithme moderne d'apprentissage par renforcement particulièrement utilisé pour sa stabilité et ses bonnes performances dans des environnements variés.

Le principe général de PPO est d'apprendre progressivement une politique :

$$\pi(a|s)$$

associant à chaque état s une action a . Dans le cas de `CartPole`, l'agent apprend progressivement à choisir les actions permettant de maintenir le pendule en équilibre le plus longtemps possible.

L'entraînement est réalisé directement sur la récompense native de l'environnement `CartPole-v1`. Une fois l'apprentissage terminé, l'agent atteint des scores élevés proches de la performance maximale autorisée par l'environnement.

Cet agent entraîné constitue alors notre expert.

III.3 Génération des démonstrations

Une fois l'agent expert obtenu, plusieurs épisodes sont générés afin de construire un ensemble de trajectoires expertes.

Chaque épisode correspond à une trajectoire :

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$$

où :

- s_t représente l'état observé à l'instant t ;
- a_t représente l'action choisie par l'expert ;
- T représente la fin de l'épisode.

Dans notre implémentation, chaque transition enregistrée contient :

$$(s_t, a_t, r_t, s_{t+1}, done_t)$$

avec :

- s_t : état courant ;
- a_t : action réalisée ;
- r_t : récompense native de l'environnement ;
- s_{t+1} : état suivant ;
- $done_t$: indicateur de fin d'épisode.

Même si la récompense réelle de l'environnement n'est pas utilisée directement lors de l'apprentissage inverse, elle est conservée afin de faciliter certaines phases d'analyse et de validation expérimentale.

III.4 Filtrage des trajectoires expertes

Toutes les trajectoires produites par l'agent ne sont pas nécessairement conservées. En effet, l'Inverse Reinforcement Learning repose sur l'hypothèse que les démonstrations correspondent à des comportements optimaux ou quasi-optimaux.

Pour cette raison, seules les trajectoires atteignant un score minimal fixé expérimentalement sont conservées dans le dataset final. Dans le cadre de `CartPole`, nous choisissons de ne garder que les épisodes atteignant un score supérieur ou égal à 450.

Ce filtrage permet :

- d'éviter l'introduction de comportements sous-optimaux ;
- de réduire l'ambiguïté lors de l'apprentissage de la récompense ;
- de fournir un ensemble cohérent de démonstrations expertes.

Les trajectoires retenues constituent alors la base de données utilisée dans les étapes suivantes du projet pour apprendre différentes fonctions de récompense.

III.5 Construction du dataset final

Les trajectoires expertes retenues sont finalement regroupées dans un dataset contenant l'ensemble des transitions observées lors des épisodes experts.

Ce dataset contient notamment :

- les états ;
- les actions ;
- les états suivants ;
- les indicateurs de fin d'épisode.

Ces données serviront ensuite à entraîner différents modèles de récompense dans le cadre de l'Inverse Reinforcement Learning.

L'objectif sera alors de construire une fonction :

$$r_{\theta}(s, a)$$

capable d'attribuer des récompenses élevées aux comportements proches des démonstrations expertes et des récompenses plus faibles aux comportements non experts.

IV Modélisation de la fonction de récompense

Nous disposons désormais d'un ensemble de démonstrations expertes construites à partir d'un agent PPO préalablement entraîné sur l'environnement `CartPole`. Ces démonstrations constituent la base de travail permettant d'aborder le véritable problème de l'Inverse Reinforcement Learning : apprendre une fonction de récompense cohérente avec le comportement observé.

L'objectif n'est plus ici d'apprendre directement une politique, mais de construire une fonction :

$$r_{\theta}(s)$$

capable d'attribuer des récompenses élevées aux comportements compatibles avec ceux de l'expert. Dans notre implémentation, nous faisons le choix de modéliser une récompense dépendant uniquement de l'état s , et non explicitement de l'action a . Ce choix simplifie le problème et semble raisonnable dans le cas de `CartPole`, où l'objectif principal consiste à maintenir le système dans des états stables.

Une fois cette récompense apprise, elle est utilisée comme signal d'apprentissage pour entraîner un nouvel agent par Reinforcement Learning classique. La qualité de la récompense apprise ne se mesure donc pas uniquement à sa capacité à distinguer les trajectoires expertes, mais surtout à sa capacité à permettre l'apprentissage d'une politique performante.

Cette étape constitue le cœur du projet, car elle correspond au passage :

comportement expert $\rightarrow$ fonction de récompense

alors qu'en Reinforcement Learning classique, le problème est généralement :

fonction de récompense $\rightarrow$ politique optimale.

IV.1 Difficulté de la modélisation

L'apprentissage d'une fonction de récompense constitue un problème difficile. Contrairement à un problème supervisé classique, il n'existe pas nécessairement une unique récompense expliquant le comportement observé.

Dans le cas de `CartPole`, plusieurs formulations peuvent conduire à un comportement performant :

- maintenir le pendule proche de la verticale ;
- limiter les vitesses angulaires ;
- éviter les états de chute ;
- maximiser la durée de survie de l'épisode.

Toutes ces récompenses peuvent produire des politiques proches. Le problème est donc sous-déterminé : plusieurs fonctions de récompense différentes peuvent expliquer les mêmes démonstrations expertes.

De plus, une difficulté supplémentaire est apparue expérimentalement : une fonction de récompense capable de séparer correctement des états experts et des états non experts ne constitue pas nécessairement un bon signal d'apprentissage pour un agent RL. Une récompense utile doit également être progressive, stable, et exploitable par l'algorithme d'apprentissage.

IV.2 Première approche : apprentissage discriminatif état par état

Dans un premier temps, nous avons formulé l'apprentissage de la récompense comme un problème de discrimination entre :

- les états provenant des trajectoires expertes ;
- les états provenant de trajectoires non expertes.

L'idée était d'entraîner un modèle attribuant un score élevé aux états experts et un score faible aux états non experts. Plusieurs familles de modèles ont été testées :

- une récompense linéaire ;
- une récompense quadratique ;
- une récompense représentée par un réseau de neurones ;

Cette approche présente l'avantage d'être simple à mettre en œuvre et de fournir une première baseline. Cependant, les résultats ont montré une limite importante : les modèles les plus capables de discriminer les états experts des états non experts ne sont pas nécessairement ceux qui permettent d'entraîner les meilleurs agents.

La récompense linéaire obtient ponctuellement des performances relativement élevées, mais avec une variance extrêmement importante. Les récompenses quadratique et neuronale restent quant à elles nettement insuffisantes. Ces résultats suggèrent qu'une bonne séparation expert/non-expert ne garantit pas l'obtention d'une récompense robuste pour l'apprentissage par renforcement.

Modèle de récompense	Score moyen réel	Écart-type
Récompense linéaire	206.00	195.78
Récompense quadratique	9.38	0.70
Réseau de neurones	19.40	3.21

TABLE 1 – Performance des agents PPO entraînés avec les récompenses apprises par discrimination état par état.

Les résultats obtenus montrent que l'apprentissage discriminatif état par état reste insuffisant pour reconstruire une récompense robuste. Néanmoins, un comportement intéressant apparaît : la récompense linéaire obtient un score moyen supérieur aux modèles plus complexes.

Ce résultat peut s'expliquer par la nature de l'environnement CartPole. La récompense native de CartPole est constante et indépendante de l'état. Une modélisation simple semble donc plus adaptée qu'un modèle fortement non linéaire susceptible d'apprendre des frontières de décision artificielles entre états experts et non experts.

Cependant, l'écart-type très important observé pour la récompense linéaire indique une forte instabilité des politiques apprises. Malgré des épisodes parfois performants, la récompense ne fournit pas un signal suffisamment robuste pour reproduire systématiquement le comportement expert.

IV.3 Limite de l'approche discriminative

L'approche précédente présente une faiblesse méthodologique : elle apprend une récompense localement, état par état, sans raisonner directement au niveau des trajectoires complètes.

Or, dans le cadre de l'IRL, les démonstrations expertes sont précisément des trajectoires :

$$\tau_E = (s_0, a_0, s_1, a_1, \dots, s_T)$$

et non une simple collection indépendante d'états. Il est donc plus naturel de chercher à apprendre une récompense telle que les trajectoires expertes obtiennent un meilleur retour que les trajectoires produites par l'agent courant.

Cette réflexion nous a conduits à abandonner l'approche discriminative état par état comme méthode principale, pour retenir une approche par préférence de trajectoires.

V Méthode retenue : apprentissage par préférence de trajectoires

V.1 Principe

La méthode finalement retenue consiste à apprendre une fonction de récompense $r_\theta(s)$ telle que les trajectoires expertes obtiennent un retour supérieur aux trajectoires produites par l'agent courant.

Pour une trajectoire τ , on définit le retour appris :

$$R_\theta(\tau) = \frac{1}{T} \sum_{t=0}^T r_\theta(s_t)$$

Nous utilisons ici une moyenne plutôt qu'une somme afin d'éviter que le modèle apprenne uniquement à favoriser les trajectoires longues. L'objectif est d'apprendre une notion de qualité moyenne des états visités, et non simplement de réintroduire indirectement la récompense native de CartPole.

À chaque itération, nous disposons :

- d'un ensemble de trajectoires expertes τ_E ;

— d'un ensemble de trajectoires générées par l'agent courant τ_π .

La fonction de récompense est alors entraînée pour satisfaire :

$$R_\theta(\tau_E) > R_\theta(\tau_\pi)$$

Ce principe est implémenté à l'aide d'une fonction de perte de type ranking loss :

$$\mathcal{L}(\theta) = \max(0, m - (R_\theta(\tau_E) - R_\theta(\tau_\pi)))$$

où m est une marge positive. Cette marge impose que le retour appris d'une trajectoire experte soit supérieur à celui d'une trajectoire agent d'au moins m .

V.2 Boucle d'apprentissage

La boucle complète est la suivante :

1. charger les trajectoires expertes ;
2. initialiser un modèle de récompense r_θ ;
3. collecter des trajectoires produites par l'agent courant ;
4. mettre à jour r_θ afin que les trajectoires expertes soient préférées aux trajectoires agent ;
5. entraîner PPO avec la récompense apprise ;
6. collecter de nouvelles trajectoires agent ;
7. répéter le processus.

Cette procédure est plus proche de l'esprit du papier étudié que la première approche discriminative, car la récompense n'est pas apprise une fois pour toutes à partir d'exemples fixes. Elle est progressivement ajustée en fonction des trajectoires produites par la politique courante.

La vraie récompense de `CartPole` n'est pas utilisée pour entraîner la fonction de récompense. Elle est uniquement utilisée à la fin, comme métrique d'évaluation indépendante, afin de vérifier si l'agent entraîné avec la récompense apprise atteint effectivement une bonne performance dans l'environnement.

V.3 Architecture de la récompense

La fonction de récompense est représentée par un réseau de neurones simple :

$$r_\theta(s) = f_\theta(s)$$

où $s = (x, \dot{x}, \theta, \dot{\theta})$ est l'état de l'environnement. Le réseau prend en entrée les quatre variables de l'état et produit un score scalaire.

Contrairement aux premières expériences, nous n'utilisons pas de fonction d'activation de type sigmoid ou tanh en sortie. En effet, une sortie bornée entre 0 et 1 peut introduire une récompense positive à chaque pas de temps, ce qui revient implicitement à favoriser la durée de l'épisode et donc à réintroduire une information proche de la récompense native de `CartPole`. Pour éviter cela, la sortie du modèle reste non bornée, puis elle est simplement tronquée lors de son utilisation par PPO afin de stabiliser l'apprentissage.

VI Résultats expérimentaux

VI.1 Résultats de la boucle par préférence de trajectoires

La méthode par préférence de trajectoires a permis d'obtenir une amélioration progressive de la politique entraînée avec la récompense apprise.

Après dix itérations, les résultats obtenus sont les suivants :

- longueur moyenne des épisodes : 490.75 ± 17.32 ;
- score moyen selon la récompense native de `CartPole`, utilisée uniquement pour analyse : 484.30 ± 30.31 .

Ces résultats montrent que la récompense apprise fournit un signal d'apprentissage suffisamment pertinent pour entraîner une politique proche de celle de l'expert.

Méthode	Score moyen réel	Écart-type
Récompense linéaire	206.00	195.78
Récompense quadratique	9.38	0.70
Réseau de neurones	19.40	3.21
Reward par préférence de trajectoires	484.30	30.31

TABLE 2 – Comparaison des performances obtenues selon la méthode d'apprentissage de la récompense.

L'écart entre les deux approches est net. L'apprentissage par préférence de trajectoires permet d'obtenir une politique très proche du niveau expert, tandis que les approches discriminatives état par état restent insuffisantes.

VI.2 Réutilisation de la récompense finale

Afin de vérifier que la récompense finale apprise n'est pas uniquement utile dans la boucle itérative, nous avons ensuite sauvegardé le modèle de récompense final, puis entraîné un nouvel agent PPO depuis zéro en utilisant uniquement cette récompense rechargée.

Le nouvel agent atteint :

$$278.15 \pm 18.47$$

sur la récompense native de `CartPole`, utilisée uniquement comme métrique d'évaluation.

Ce résultat est inférieur au score obtenu dans la boucle itérative complète, mais il reste nettement supérieur à une politique aléatoire. Il montre donc que la récompense apprise est bien réutilisable, même si elle ne généralise pas parfaitement lorsqu'elle est utilisée seule pour entraîner un nouvel agent depuis zéro.

Cette différence suggère que la boucle itérative apprend à la fois :

- une fonction de récompense informative ;
- une politique progressivement adaptée à cette récompense.

La récompense finale contient donc bien un signal utile, mais la co-adaptation progressive entre la politique et la récompense joue également un rôle important dans la performance finale.

VI.3 Analyse

Les expériences mettent en évidence plusieurs points importants.

Premièrement, une récompense capable de distinguer des états experts et non experts ne suffit pas nécessairement à entraîner une bonne politique. Cette observation est centrale, car elle montre que l'apprentissage d'une récompense ne peut pas être évalué uniquement comme un problème de classification.

Deuxièmement, raisonner au niveau des trajectoires semble plus pertinent dans ce cadre. En imposant que les trajectoires expertes obtiennent un retour appris supérieur aux trajectoires de l'agent courant, la récompense apprise fournit un signal plus directement lié au comportement global.

Enfin, la vraie récompense de `CartPole` n'a été utilisée que comme métrique finale d'évaluation. L'apprentissage de la récompense repose sur les démonstrations expertes et sur les trajectoires produites par l'agent courant, ce qui conserve la logique générale de l'Inverse Reinforcement Learning.

VII Conclusion

Ce projet avait pour objectif de mettre en œuvre une chaîne simplifiée d'Inverse Reinforcement Learning sur l'environnement `CartPole`. Nous avons d'abord entraîné un agent expert avec PPO, puis utilisé ses trajectoires comme démonstrations expertes.

Une première approche consistant à apprendre une récompense discriminative état par état a permis d'obtenir des résultats contrastés. La récompense linéaire produit parfois des comportements relativement performants, mais avec une très forte variance. Les modèles quadratiques et neuronaux obtiennent quant à eux des performances nettement inférieures. Ces résultats suggèrent qu'une bonne capacité de discrimination entre états experts et non experts ne garantit pas l'obtention d'un signal d'apprentissage pertinent pour le Reinforcement Learning.

La méthode finalement retenue repose sur une préférence de trajectoires. Le modèle de récompense est entraîné pour attribuer un retour plus élevé aux trajectoires expertes qu'aux trajectoires produites par l'agent courant. Cette approche a permis d'obtenir un agent atteignant un score moyen de 484.30, proche de la performance maximale de `CartPole-v1`.

La réutilisation de la récompense finale pour entraîner un nouvel agent depuis zéro donne un score moyen de 278.15, montrant que la récompense apprise contient bien un signal utile, même si la boucle itérative complète reste plus performante.

Ces résultats illustrent une idée importante de l'IRL : la qualité d'une récompense apprise ne se juge pas uniquement à sa capacité à reconnaître les démonstrations expertes, mais surtout à sa capacité à produire, après optimisation par RL, une politique performante.

Références

- [1] Pieter ABBEEL et Andrew Y. NG. "Apprenticeship Learning via Inverse Reinforcement Learning". In : *Proceedings of the 21st International Conference on Machine Learning (ICML)*. 2004.
- [2] FARAMA FOUNDATION. *Gymnasium Documentation*. <https://gymnasium.farama.org/>. 2023.

- [3] Chelsea FINN, Sergey LEVINE et Pieter ABBEEL. “Guided Cost Learning : Deep Inverse Optimal Control via Policy Optimization”. In : *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016.
- [4] Volodymyr MNIH, Koray KAVUKCUOGLU, David SILVER et al. “Human-level Control through Deep Reinforcement Learning”. In : *Nature* 518.7540 (2015), p. 529-533.
- [5] Andrew Y. NG et Stuart RUSSELL. “Algorithms for Inverse Reinforcement Learning”. In : *Proceedings of the 17th International Conference on Machine Learning (ICML)*. 2000.
- [6] David SILVER, Aja HUANG, Chris J. MADDISON et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search”. In : *Nature* 529.7587 (2016), p. 484-489.
- [7] Richard S. SUTTON et Andrew G. BARTO. “Reinforcement Learning : An Introduction”. In : *MIT Press* (2018).
- [8] Brian D. ZIEBART. “Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy”. Thèse de doct. Carnegie Mellon University, 2010.
- [9] Brian D. ZIEBART et al. “Maximum Entropy Inverse Reinforcement Learning”. In : *Proceedings of the AAAI Conference on Artificial Intelligence*. 2008.